

```
216      4      245      61.2      39.1
self._on_scrolled_win_change)
```

In the above code, line no 213, `vadjustment = self.view.get_vadjustment()`, is called too many times, which makes the process slower than expected. After caching (initialising) it in the `init` function, we get the following result tested under the same condition. You can see that there is a significant improvement in the results (Figure 2).

Wrote profile results to `gnome-music.lprof`
Timer unit: 1e-06 s

```
File: ./gnomemusic/view.py
Function: _connect_view at line 211
Total time: 0.000466 s
```

```
Line # Hits Time Per Hit % Time Line Contents
=====
211                                     @profile
212         def _connect_view(self):
213 4      86      21.5 18.5 self._adjustmentValueId =
vadjustment.connect(
214 4      161     40.2 34.5 'value-changed',
215 4      219     54.8 47.0 self._on_scrolled_win_change)
```

Understanding the output

Here is an analysis of the output shown in the above snippet.

- **Function:** Displays the name of the function that is profiled and its line number.
- **Line#:** The line number of the code in the respective file.
- **Hits:** The number of times the code in the corresponding line was executed.
- **Time:** Total amount of time spent in executing the line in 'Timer unit' (i.e., 1e-06s here). This may vary from system to system.
- **Per hit:** The average amount of time spent in executing the line once in 'Timer unit'.
- **% time:** The percentage of time spent on a line with respect

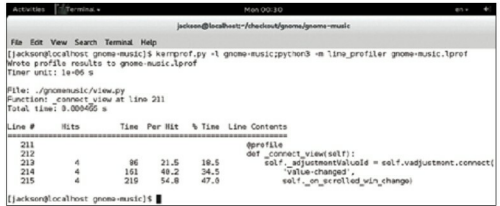


Figure 2: Optimised code line_profiler output

to the total amount of recorded time spent in the function.

- **Line content:** It displays the actual source code.



Note: If you make changes in the source code you need to run the `kernprof` and `line_profiler` again in order to profile the updated code and get the latest results.

Advantages

`Line_profiler` helps us to profile our code line-by-line, giving the number of hits, time taken for each hit and **%time**. This helps us to understand which part of our code is running slow. It also helps in testing large projects and the time spent by modules to execute a particular function. Using this data, we can commit changes and improve our code to build faster and better programs. 

References

- [1] http://pythonhosted.org/line_profiler/
- [2] http://jacksonisaac.wordpress.com/2013/09/08/using-line_profiler-with-python/
- [3] https://pypi.python.org/pypi/line_profiler
- [4] https://bitbucket.org/robertkem/line_profiler
- [5] https://bitbucket.org/kmike/line_profiler

By: Jackson Isaac

The author is an active open source contributor to projects like `gnome-music`, `Mozilla Firefox` and `Mozillians`. Follow him on jacksonisaac.wordpress.com or email him at jacksonisaac2008@gmail.com

© TechnoMail
Enterprise Email Solutions

© TechnoMail Cloud
Zero Capital Email Solution

TechnoInfotech®

info@technoinfotech.com
www.technoinfotech.com

© High Volume SMTP Solutions
(Hosted and on-Premise)

© Business Promotion Mailing
(Hosted and on-Premise)